# Documentação do Projeto

## Sistema de Controle Financeiro Pessoal

Projeto de aprendizagem em PHP + MySQL, com possibilidade de evolução para uma aplicação web completa.

**Objetivo:** construir uma ferramenta realmente utilizável no dia a dia e, ao mesmo tempo, aprender PHP de forma progressiva, passando por CRUD, banco de dados, autenticação, sessões, segurança, relatórios, APIs e arquitetura.

# 1. Visão geral

O sistema permitirá registrar, organizar e analisar receitas e despesas pessoais. O usuário terá uma conta própria, poderá cadastrar lançamentos, categorias, contas e cartões, acompanhar o saldo e visualizar relatórios mensais. A aplicação deve ser responsiva e funcionar bem em computador e celular.

# 2. Funcionalidades principais

• Cadastro, login, logout e recuperação de senha.
• Dashboard com saldo, receitas, despesas e resumo do mês.
• CRUD de receitas e despesas.
• Categorias personalizáveis.
• Contas financeiras.
• Cartões de crédito e faturas.
• Lançamentos parcelados e recorrentes.
• Filtros por período, categoria, conta e tipo.
• Busca de lançamentos.
• Relatórios mensais e anuais.
• Gráficos de gastos por categoria e evolução do saldo.
• Metas financeiras.
• Exportação CSV.
• Configurações do usuário.
• Auditoria básica de alterações importantes.

# 3. Fluxo de uso

1. O usuário cria uma conta e entra no sistema.
2. Configura suas contas e categorias.
3. Registra uma receita ou despesa.
4. O sistema atualiza os totais e o saldo disponível.
5. O dashboard mostra a situação financeira do período selecionado.
6. O usuário filtra os dados e consulta relatórios.
7. Para despesas parceladas, o sistema gera as parcelas futuras.
8. No fechamento do mês, o usuário analisa onde gastou e compara com meses anteriores.

# 4. Modelo de dados sugerido

**users**: id, name, email, password_hash, created_at, updated_at.
**accounts**: id, user_id, name, type, initial_balance, active.
**categories**: id, user_id, name, type, active.
**transactions**: id, user_id, account_id, category_id, type, amount, description, transaction_date, status, created_at, updated_at.
**credit_cards**: id, user_id, name, limit_amount, closing_day, due_day, active.
**installments**: id, transaction_id, installment_number, total_installments, due_date, amount, status.
**recurring_transactions**: id, user_id, account_id, category_id, type, amount, frequency, next_date, active.
**goals**: id, user_id, name, target_amount, current_amount, deadline, status.
**audit_logs**: id, user_id, action, entity, entity_id, created_at.

Um usuário possui várias contas, categorias, transações, cartões e metas. Uma transação pertence a uma conta e pode pertencer a uma categoria.

# 5. Regras de negócio

• Receita aumenta o saldo; despesa reduz o saldo.
• Valores monetários devem ser armazenados como DECIMAL, não float.
• Toda transação deve pertencer ao usuário autenticado.
• Despesas parceladas devem gerar parcelas vinculadas ao lançamento original.
• Transações futuras não devem reduzir o saldo realizado atual.
• O sistema pode oferecer saldo realizado e saldo projetado.
• Dados importantes podem ser arquivados em vez de apagados definitivamente.

# 6. Dashboard

O painel inicial deve apresentar saldo total, receitas do mês, despesas do mês, resultado do mês, contas, próximas parcelas, maiores categorias de gasto e evolução mensal. Um filtro de período deve atualizar os indicadores. Gráficos podem ser feitos com Chart.js.

# 7. Tecnologias

Backend: PHP 8.x, preferencialmente com arquitetura MVC.
Banco: MySQL ou MariaDB.
Frontend: HTML5, CSS3 e JavaScript; Bootstrap ou CSS próprio.
Banco via PDO com prepared statements.
Gráficos: Chart.js.
Dependências: Composer.
Versionamento: Git/GitHub.
Ambiente local: XAMPP, Laragon ou Docker.
Produção: hospedagem PHP + MySQL ou VPS.

# 8. Segurança

• Senhas com password_hash() e password_verify().
• Prepared statements para evitar SQL Injection.
• Validação dos dados no servidor; filter_input() pode auxiliar na leitura de entradas externas.
• Escape de saída HTML para reduzir XSS.
• Regeneração do ID de sessão após login.
• Proteção CSRF em formulários que alteram dados.
• Autorização por user_id em todas as consultas.
• HTTPS em produção.

A documentação oficial do PHP recomenda password_hash() para criar hashes fortes e informa que PASSWORD_DEFAULT pode mudar de algoritmo; por isso, uma coluna de até 255 caracteres é uma boa escolha. ■cite■turn0search9■

# 9. APIs e integrações sugeridas

**Banco Central do Brasil:** o Portal de Dados Abertos oferece APIs, JSON e OData, incluindo conjuntos relacionados à Selic e expectativas de mercado. Você pode criar uma área de indicadores econômicos ou usar esses dados em simuladores. ■cite■turn0search2■turn0search6■

**ViaCEP:** útil para preenchimento automático de endereço, caso o cadastro venha a armazenar endereço.

**API própria:** depois do núcleo estar pronto, crie uma API REST do próprio sistema. Exemplos: GET /api/transactions, POST /api/transactions, PUT /api/transactions/{id}, DELETE /api/transactions/{id}. Isso ensina HTTP, JSON, autenticação e integração entre frontend e backend.

Recomendação: primeiro implemente o núcleo financeiro; APIs externas entram depois.

## 10. Relatórios

Relatórios recomendados: gastos por categoria, receitas x despesas, evolução do saldo, gastos por conta, despesas recorrentes, parcelas futuras, comparação entre meses e cumprimento de metas. Use SUM(), GROUP BY, WHERE e JOIN para gerar os dados.

## 11. Estrutura de pastas

```
project/
■■■ app/
■ ■■■ Controllers/
■ ■■■ Models/
■ ■■■ Services/
■ ■■■ Views/
■■■ config/
■■■ database/
■ ■■■ migrations/
■■■ public/
■ ■■■ index.php
■ ■■■ css/
■ ■■■ js/
■■■ routes/
■■■ storage/
■■■ tests/
■■■ vendor/
■■■ .env
■■■ composer.json
■■■ README.md
```

## 12. Roadmap de aprendizagem

**V1:** páginas PHP, formulários, GET/POST, condicionais, loops e funções.
**V2:** CRUD de transações.
**V3:** MySQL/PDO e prepared statements.
**V4:** Models, Controllers, Views e rotas.
**V5:** login, sessões e autorização.
**V6:** categorias, contas e JOINs.
**V7:** dashboard e gráficos.
**V8:** parcelamentos e faturas.
**V9:** recorrências.
**V10:** segurança e tratamento de erros.
**V11:** API REST e JSON.
**V12:** integrações externas.
**V13:** deploy, HTTPS, backups e logs.

## 13. SQL que você deverá aprender

SELECT, INSERT, UPDATE, DELETE, WHERE, ORDER BY, LIMIT, LIKE, BETWEEN, JOIN, LEFT JOIN, GROUP BY, HAVING, SUM, COUNT, AVG, MIN, MAX, CASE, subconsultas, índices e chaves estrangeiras.

## 14. Exemplo de API interna

```
GET /api/transactions
GET /api/transactions/15
POST /api/transactions
PUT /api/transactions/15
DELETE /api/transactions/15

POST /api/auth/login
POST /api/auth/logout
GET /api/dashboard/monthly
```

As respostas devem ser JSON e utilizar códigos HTTP adequados, como 200, 201, 400, 401, 403, 404 e 500.

## 15. Melhorias futuras

• PWA para celular.
• Importação de extrato CSV.
• Classificação automática de categorias.
• Backup automático.
• Modo escuro.
• Multi-moeda.
• Orçamento familiar.
• Notificações de vencimento.
• Previsão de despesas.
• Integração com Open Finance somente em uma etapa avançada e usando um provedor adequado.

## 16. Critérios de conclusão

A primeira versão está completa quando o usuário conseguir criar conta, fazer login, cadastrar contas e categorias, registrar receitas/despesas, editar/excluir lançamentos, consultar saldo, filtrar períodos e visualizar gráficos. A segunda versão deve acrescentar cartões, parcelas, recorrências, metas, relatórios e API.

## 17. Fontes técnicas

Documentação oficial do PHP para password_hash e filter_input. O Portal de Dados Abertos do Banco Central disponibiliza APIs, JSON e OData para conjuntos públicos, e documenta a plataforma Olinda para acesso aos dados. ■cite■turn0search0■turn0search4■

## Estratégia recomendada

Construa o projeto incrementalmente. Não tente implementar tudo de uma vez. Cada versão deve introduzir um conceito novo de PHP, SQL ou arquitetura. Assim, o sistema cresce junto com seu conhecimento e permanece útil desde as primeiras versões.